

DRAFT PROJECT REPORT

INSIGHT MART
20TH MARCH 2025

Module: Data Science Project Life
Team: Group A8

UNIVERSITY OF
WESTMINSTER 

Table of Contents

Executive Summary	4
Overview of KJ Marketing's Challenge.	4
Project Objective	4
Data Observation.....	5
Train dataset	5
Test dataset.....	5
Data Cleaning and Pre-processing.....	6
Train dataset	6
Data Cleaning.....	6
Missing Values	7
Outliers.....	8
Test dataset.....	9
Exploratory Data Analysis (EDA)	10
Train data set	10
Univariate Analysis.....	10
Multivariate Analysis.....	14
Customer segmentation Analysis and Labelling	17
How ?	18
Bulk buyers (cluster 1)	18
Health-Conscious Buyers (Cluster2)	18
Luxury Seekers (Cluster 3)	18
Budget-Conscious Shoppers (Cluster 4)	18
Occasional Shoppers (Clusters 5)	19
Everyday Shoppers (Cluster6)	19
Feature Engineering.....	20
Encoding Strategies	20
Standardization & Normalization	20
Features.....	20
Splitting.....	21

Model Development	22
Algorithms Considered	22
Model Selection & Performance	22
Best Model : Logistic Regression	22
Why choose logistic regression model to be the best model?	23
Challenges & Solutions	24
Overfitting	24
Computational Constraints	24
Marketing Strategy Recommendations	25
Marketing Strategy Recommendations based on customer segmentations	25
1. Bulk Buyers	25
2. Health-Conscious Buyers	25
3. Luxury Seekers	25
4. Budget-Conscious	26
5. Occasional Shoppers	26
6. Everyday shoppers	26
Project Plan	27
Work Breakdown Structure	27
Gantt chart	28
Project team	28
GitHub Link	29
Canva Link	29
TEAM MEMBERS	29

Executive Summary

Overview of KJ Marketing's Challenge.

KJ Marketing, a leading supermarket chain in Sri Lanka, has observed that the traditional marketing strategies are becoming less effective in engaging its evolving customer base. With 22 outlets serving urban and suburban regions, the company offers a wide variety of products, including luxury, fresh and dry goods. However, the lack of targeted marketing has led to a poor customer engagement and this might lead to inefficient inventory management impacting revenue and customer satisfaction. To address this Insight Mart has aimed to implement a personalized marketing strategy based on customer segmentation for KJ Marketing.

Project Objective

Our primary goal is to develop a machine learning (ML) model that accurately classifies new customers into six predefined customer segments based on their purchasing behaviour.

Specially, we aim to:

- Analyse historical sales data to understand customer purchasing behaviour.
- Implement a machine learning model to classify customers into predefined segments.
- Optimize the model for high accuracy and robust generalization.
- Extract business Insights that will help KJ Marketing enhance customer engagement, optimize marketing campaigns and inventory management.

Data Observation

In order to enhance the marketing strategies Insight Mart is provided with historical sales data, which includes average monthly sales per customer. The traincsv dataset is used for training the ML model, while the test dataset is used to evaluate the model's performance.

Train dataset

This dataset contains labeled data which means it includes both input features (luxury sales, fresh sales and dry sales, outlet city) and the target variable (cluster category). The dataset contains 774154 rows and 6 columns including both numerical and categorical values.

<

There are both missing values and outliers in the dataset that were handled using the necessary imputation techniques. However no duplicates were found in the dataset.

Test dataset

The test dataset does not include the target variable cluster category because the goal is to assess how well the trained model can classify the new customers to the relevant customer segments. Test dataset has 40749 rows with the five columns except cluster category. It has no missing values or duplicates. There was one outlier in fresh sales which was later removed in pre-processing.

Data Cleaning and Pre-processing

Train dataset

Data Cleaning

First, in order to clean the dataset, all the spaces in the columns were removed. All the sales columns were converted into string data type using `.astype(str)` before replacing. This is necessary because applying `.str.replace()` directly on non string type data would cause an error. `apply(lambda x: x.str.replace(' ', '', regex=True))` function is applied to the selected columns and to remove all spaces in each column's values. `.str.replace(' ', '', regex=True)` is used. Then it ensures that the replacement operation is performed using regular expressions.

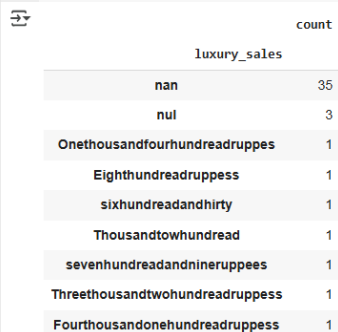
```
[ ] # Removing spaces in column values
columns_to_clean = ['luxury_sales', 'fresh_sales', 'dry_sales']

# Convert columns to string type before applying str.replace
for col in columns_to_clean:
    df[col] = df[col].astype(str)

df[columns_to_clean] = df[columns_to_clean].apply(lambda x: x.str.replace(' ', '', regex=True))

import re

# Extract rows where the 'luxury_sales' column contains strings
strings_in_luxury_sales = df['luxury_sales'][df['luxury_sales'].apply(lambda x: bool(re.match(r'[a-zA-Z]', str(x))))]
strings_in_luxury_sales.value_counts()
```



luxury_sales	count
nan	35
nul	3
Onethousandfourhundreadruppes	1
Eighthundreadruppes	1
sixhundreadandthirty	1
Thousandtowhundread	1
sevenhundreadandnineruppes	1
Threethousandtwohundreadruppes	1
Fourthousandonehundreadruppes	1

The `re` module (Regular Expressions) is imported to help match patterns in column values. Then the rows where luxury columns contains strings were extracted. Then each value in luxury sales were converted into a string type and checked if the value starts with an alphabetical character. `bool()` ensures the result is `TRUE` and if the value contains `FALSE` otherwise. Then the frequency of each unique string value is counted.

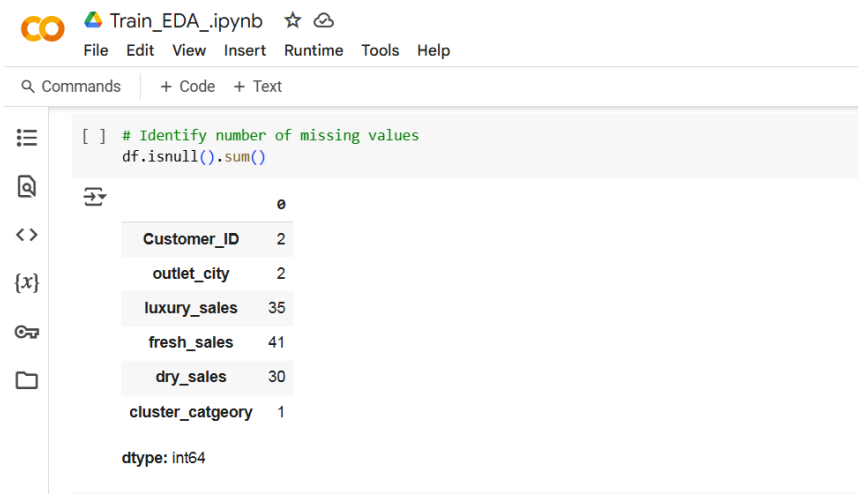
Similarly, this is performed for fresh sales and dry sales.

The recognized String values were removed and replaced with the relevant values.

```
# Remove string values and symbols
df.replace({'Onethousandfourhundredruppes': 1400,
           'Eighthundredruppes': 800,
           'sixhundredandthirty': 630,
           'Thousandtowhundred': 2200,
           'sevenhundredandnineruppes': 709,
           'Threethousandtwohundredruppes': 3200,
           'Fourthousandonehundredruppes': 4100,
           'Sixhundredandfiveruppes': 605,
           'Threethousanaandfivehundred': 3500,
           'thirteenthousandruppes': 13000,
           'Fivethousandruppes': 5000,
           'Twothousandsevenhundredruppes': 2700,|
           'Fourthousandandtwohundredruppes': 4200,
           '6\\': 6}, inplace=True)
```

Missing Values

In data observation we have identified missing values in all the columns of the traincsv, such as 2 in Customer_ID, 2 in outlet_city, 35 in luxury_sales, 41 in fresh_sales, 30 in dry_sales and 1 in cluster_catrgory.

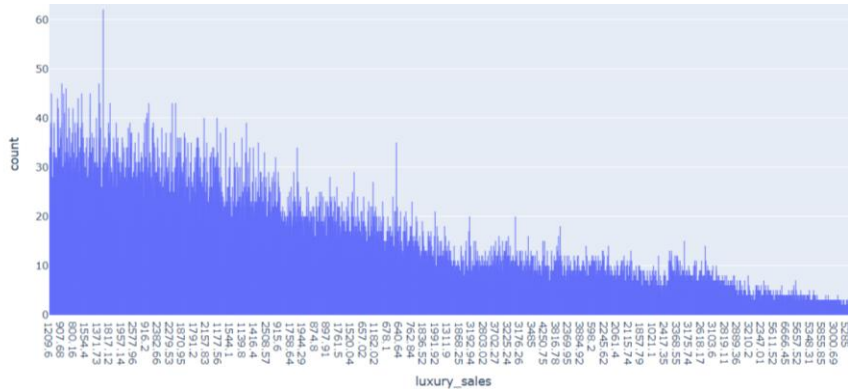


The screenshot shows a Jupyter Notebook titled 'Train_EDA_ipynb'. The code cell contains the command `df.isnull().sum()` to identify missing values. The output is a series of counts for each column: Customer_ID (2), outlet_city (2), luxury_sales (35), fresh_sales (41), dry_sales (30), and cluster_catgeory (1). The data type is listed as `dtype: int64`.

```
[ ] # Identify number of missing values
df.isnull().sum()

Customer_ID    2
outlet_city    2
luxury_sales   35
fresh_sales    41
dry_sales      30
cluster_catgeory 1
dtype: int64
```

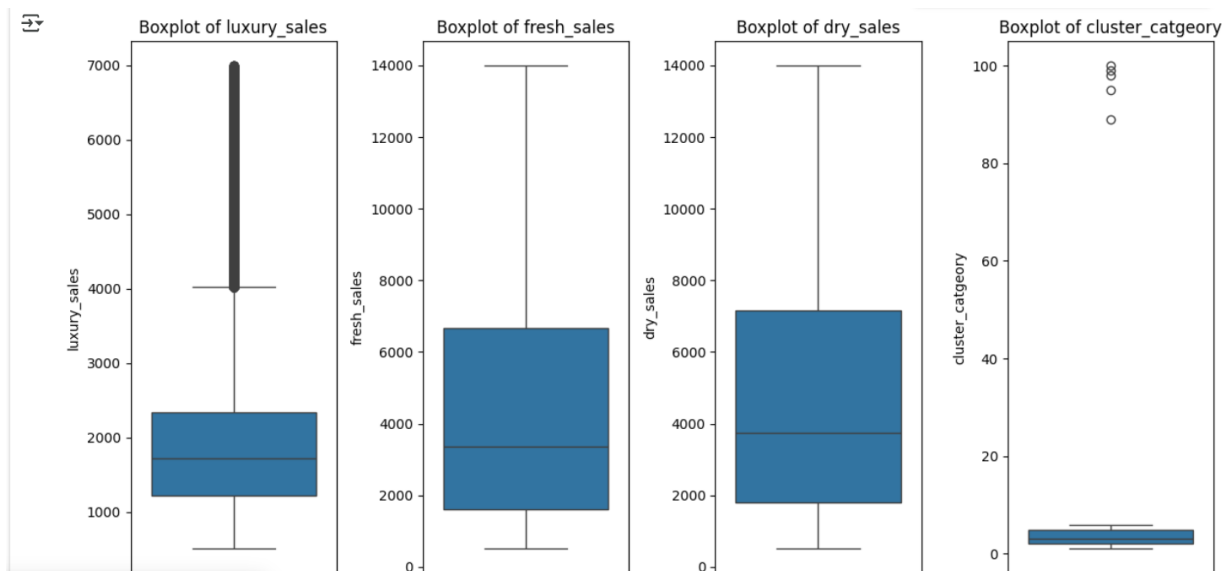
Mode and median imputation techniques were used to handle the missing values. Since all the luxury, fresh and dry sales has a right-skewed distribution, median imputation is used to handle the missing values while mode imputation is used for categorical data where the missing values of cluster_category and outlet_city are replaced with the mode of the specific columns.



Above is a histogram of one of the sales types where it is positively skewed and since median is resistant to outliers the missing values are replaced with median. Similarly all the missing values of sales columns are replaced with the respective median values.

Outliers

Boxplots were used to identify and visualize the outliers of sales columns and cluster category as follows.



Outlier Detection Function was used to identify the outliers using the Interquartile Range (IQR). The outliers those were detected in the luxury column were not considered as outliers because usually luxury goods are goods with higher prices than ordinary goods.

There were no outliers detected in fresh sales and dry sales. However, 5 outliers were detected in cluster category.


```

✓ [26] # Detect outliers in cluster_catgeory with outlier detection function
0s outliers = find_outliers_IQR(df['cluster_catgeory'])
print("number of outliers: "+ str(len(outliers)))
outliers

```

```

↕ number of outliers: 5

```

cluster_catgeory	
17	99
172456	95
172689	98
555703	100
558115	89

```
dtype: int64
```

In the initial analysis only 6 customer segments of customers were identified. Therefore the outliers identified were removed using the drop function.

Test dataset

The data cleaning was performed to the Test dataset in a similar way. First all the string values and symbols were removed and replaced similarly. Then the data types of the sales columns were converted into 'float'

```

[ ] # Remove string values and symbols
df.replace({'SevenHundreadandsixtyRuppees': 760,
           'EightHundreadandfifteenruppes': 815,
           'Ninethousandruppes': 19000,
           'ThreeThousandsevenhundreadruppees': 3700}, inplace=True)

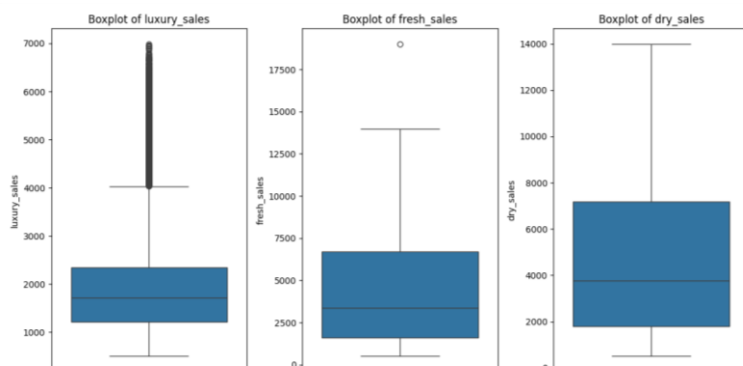
```

```

[ ] #Change data types
df['luxury_sales'] = df['luxury_sales'].astype('float')
df['fresh_sales'] = df['fresh_sales'].astype('float')
df['dry_sales'] = df['dry_sales'].astype('float')

```

No missing values or duplicates were identified in the dataset. However one outlier was detected in fresh sales.



This was removed using outlier detection function. 1841 outliers were detected in luxury sales column but just like in train dataset, since luxury goods price higher than normal goods like fresh goods or dry goods. Therefore these values are not considered as outliers.

In the outlet cities there are values with first letter capitalized and also simplified. In order to be consistent with train dataset measures will be taken in order to change them all into Capital letters.

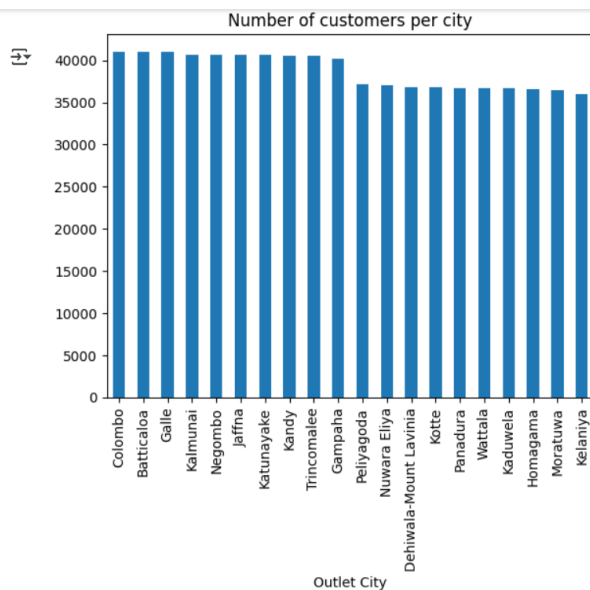
Exploratory Data Analysis (EDA)

Train data set

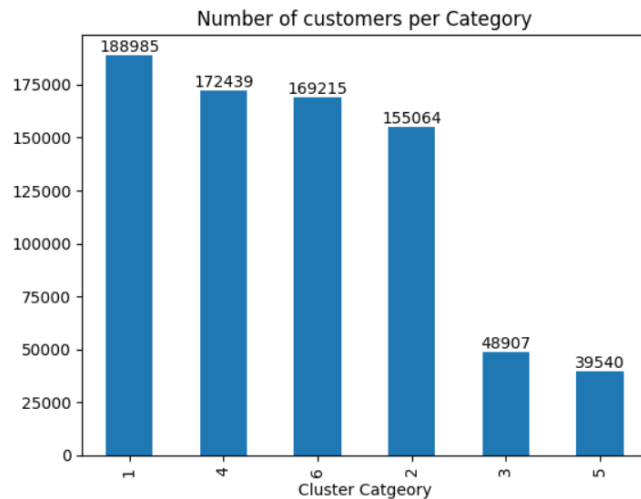
Univariate Analysis

Following EDA is performed for the better understanding of the Dataset. Univariate Analysis is performed in order to understand the distribution of the Dataset and gain insights.

Creating Bar Charts



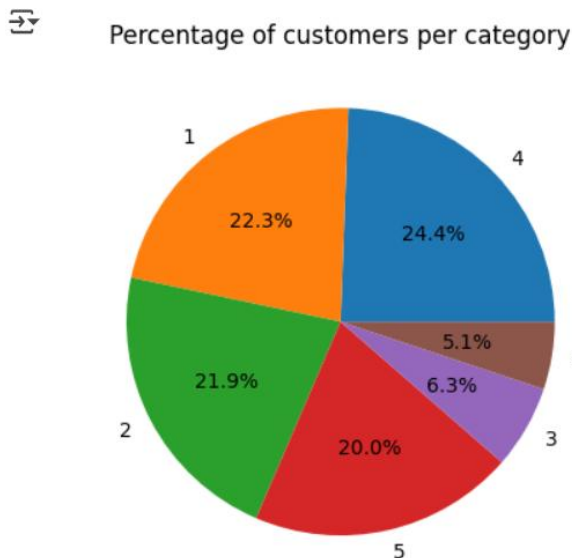
This bar chart shows the number of customers per outlet cities. Colombo, Batticaloa and Galle has the highest number of customers.



This bar graph is to show the number of customers per each cluster category. The cluster categories will be explained in detail later. However, most of the customers are bulk buyers. Cluster 6 represents everyday buyers. Cluster 3 and 5 represent luxury buyers and occasional buyers who are really low in numbers compared to the rest.

Creating Pie Charts

A pie chart is created in order to visualize this more clearly. The percentages of the cluster categories are represented in this pie chart.



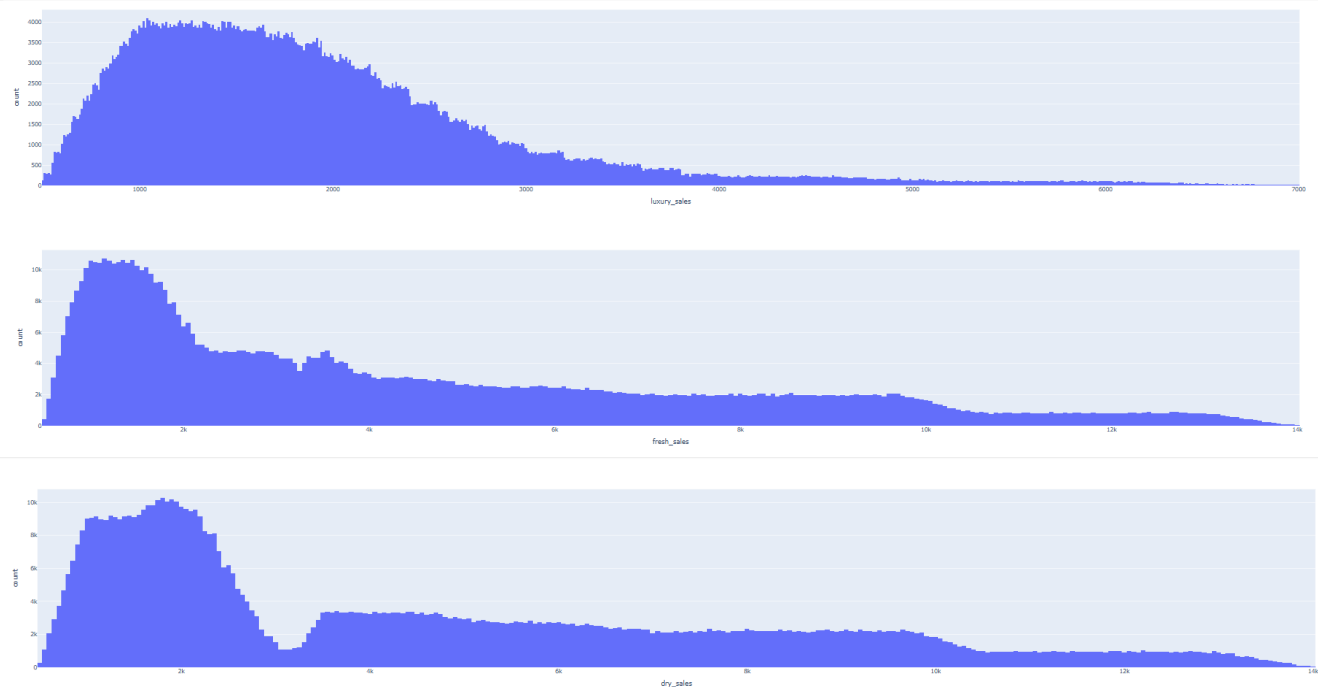
Descriptive Measures for Numerical Values

Following is a statistical analysis done in order to display the count, mean, standard deviation, minimum, 25%, 50% and 75% quartiles and the maximum of numerical values.

```
[36] # Check the statistical analysis
df.describe()
```

	Customer_ID	luxury_sales	fresh_sales	dry_sales	cluster_catgeory
count	7.741480e+05	774150.000000	774150.000000	774150.000000	774150.000000
mean	1.038708e+07	1921.959149	4428.623964	4676.244392	3.292101
std	2.234797e+05	1004.058439	3334.579088	3409.311590	1.867015
min	1.000000e+07	500.000000	500.000000	500.000000	1.000000
25%	1.019354e+07	1213.552500	1620.220000	1788.000000	2.000000
50%	1.038708e+07	1715.280000	3356.510000	3727.760000	3.000000
75%	1.058062e+07	2338.680000	6671.280000	7161.760000	5.000000
max	1.077415e+07	6999.650000	13997.900000	13999.300000	6.000000

Generating Histograms



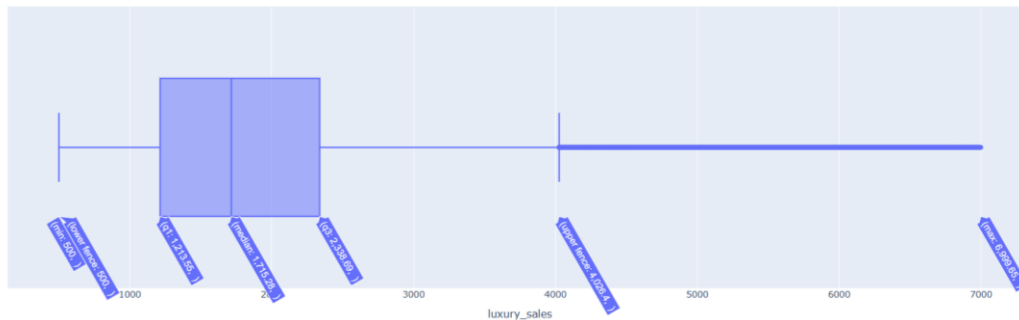
Above histograms are generated to show the distribution of the sales types of the dataset. All three histograms show right-skewed distributions. Luxury sales has a longer tail and this pattern suggests that a majority of data points have lower values than others while a few have significantly higher values. This is possible because they are luxury goods and are usually high priced.

Fresh sales has a more distinct peak possibly indicating more concentrated values than first histogram. Dry sales histogram appears to have more symmetrical distribution though it still is slightly skewed to

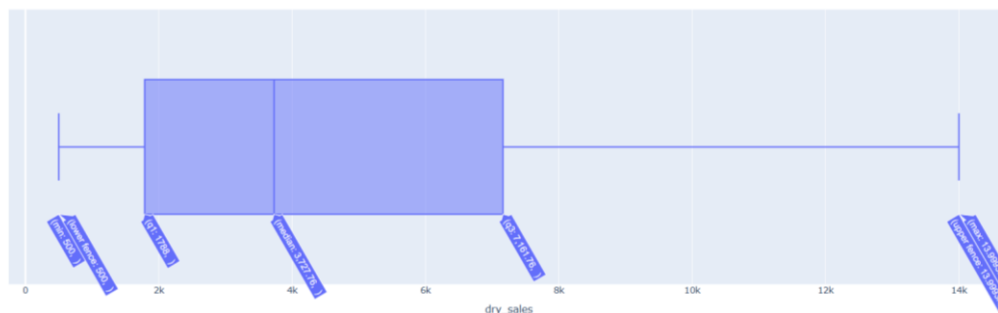
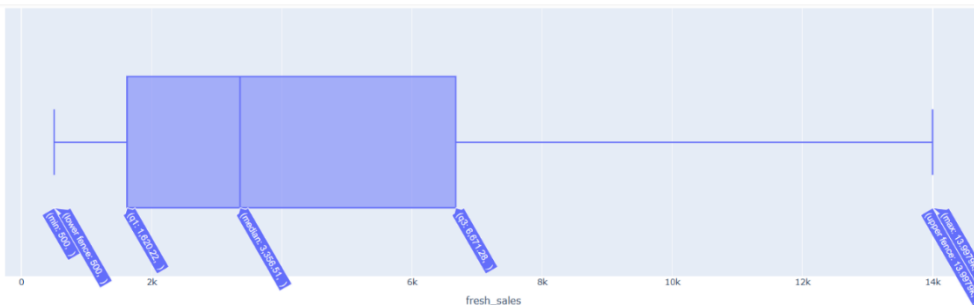
the right. It suggests that the sales are more evenly spreaded across a range of values. These histograms are useful in deciding which imputation is to be used in handling missing values.

Generating Box Plots

Boxplots were generated in order to identify statistical data and outliers.



The boxplot for the luxury sales reveals that most of the sales remain within moderate range, occasional exceptionally high sales create a long tail. According to the Boxplot 25% of the luxury sales is below Rs. 1213.55 and 75% of the sales falls approximately below Rs.2340. The median (Q2), or the 50th percentile is Rs1715.28 indicating that half of the data is below this value. The whiskers extend from 500(minimum) to the upper range limit. Similarly, the other two boxplots reveals the percentile values statistical data for fresh and dry sales which helps identifying outliers.



Multivariate Analysis

Multivariate analysis determines how variables interact with each other. This helps identifying patterns, prediction and forecasting and also in improving the decision making.

Creating Crosstabs

```
# Create a crosstab using the crosstab method  
pd.crosstab(index= df['outlet_city'], columns= df['cluster_catgeory'])
```

cluster_catgeory	1	2	3	4	5	6
outlet_city						
Batticaloa	18960	2	4901	0	2	17147
Colombo	19023	0	4959	3	2	17046
Dehiwala-Mount Lavinia	1	15648	0	17283	3902	4
Galle	19034	2	4932	1	3	17037
Gampaha	18763	1	4833	0	1	16654
Homagama	2	15464	0	17220	3951	1
Jaffna	18822	2	4906	3	2	16940
Kaduwela	1	15478	1	17360	3853	2
Kalmunai	18853	3	4831	1	1	17022
Kandy	18898	0	4843	2	0	16791

The crosstab analysis between outlet_city and cluster_category provides insights into the distribution of cluster assignments across different city outlets. Each row represents a city, while column corresponds to a cluster category with values indicating count of occurrences. For instance, Batticaloa has a significant presence in cluster 1 and 6, while Dehiwala-Mount Lavinia has a notable counts in cluster 1, 3 and 5. (The cluster names will be recognized later in this report)

Creating Pivot tables

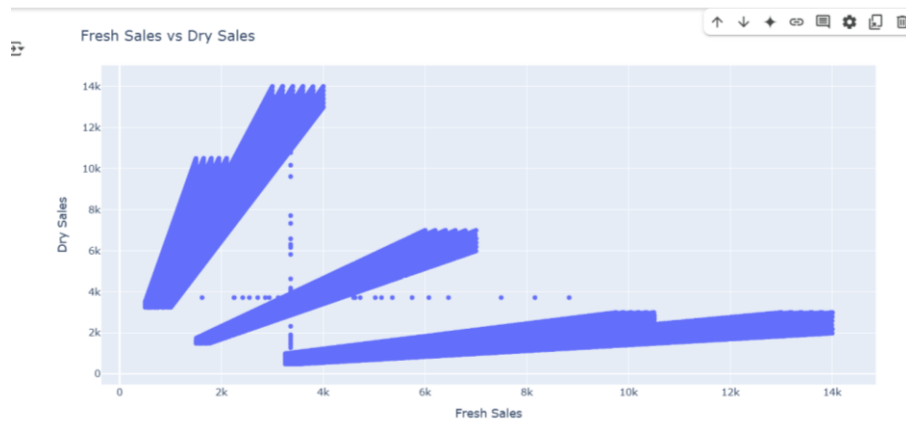
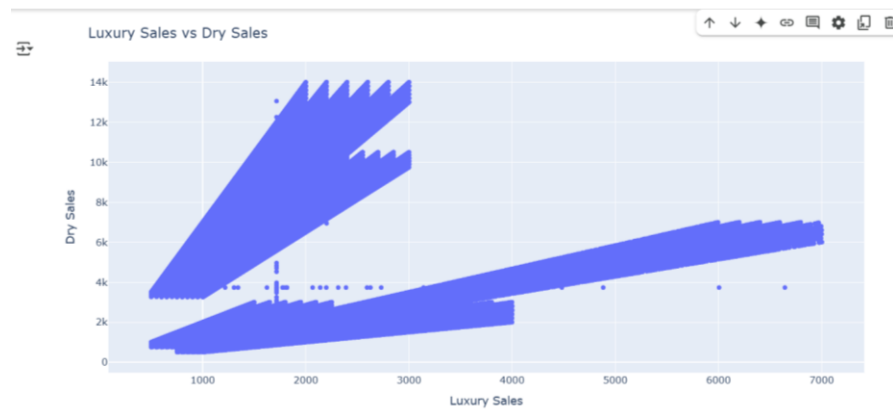
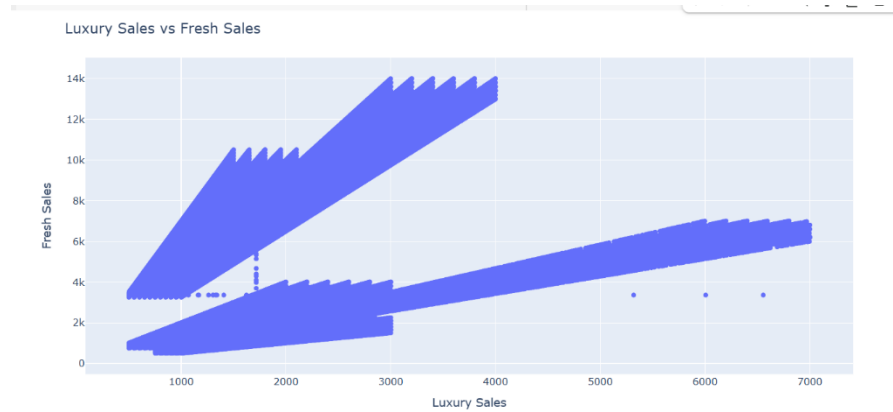
	luxury_sales
outlet_city	
Batticaloa	2127.883152
Colombo	2124.772409
Dehiwala-Mount Lavinia	1692.755487
Galle	2127.075014
Gampaha	2116.046320
Homagama	1698.885078
Jaffna	2124.764573
Kaduwela	1698.253797
Kalmunai	2114.897637
Kandy	2123.463779
Katunayake	2114.212498
Kalaniva	1691.716597

	dry_sales
outlet_city	
Batticaloa	5053.983633
Colombo	5041.190141
Dehiwala-Mount Lavinia	4245.510302
Galle	5068.087483
Gampaha	5062.512726
Homagama	4257.238822
Jaffna	5043.786248
Kaduwela	4279.756572
Kalmunai	5039.285445
Kandy	5064.294759
Katunayake	5077.631184

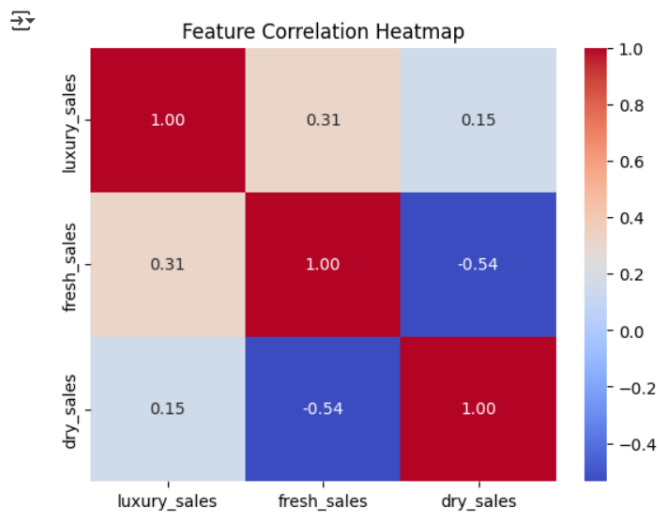
These pivot table analysis provides insights into the sales performance of average luxury and fresh products across different outlet cities. By calculating the mean sales for each city, the data highlights variations in sales trends. For instance, Batticaloa and Colombo have similar average luxury sales, around 2127 and 2124 respectively, while fresh sales are slightly higher in Batticaloa compared to Colombo. This analysis helps in identifying cities with higher or lower sales, allowing for better inventory management, marketing strategies, and business decision-making.

Creating Scatter Plots

These scatter plots shows the relationships between the three types of sales; fresh, dry and luxury, which all numerical values.



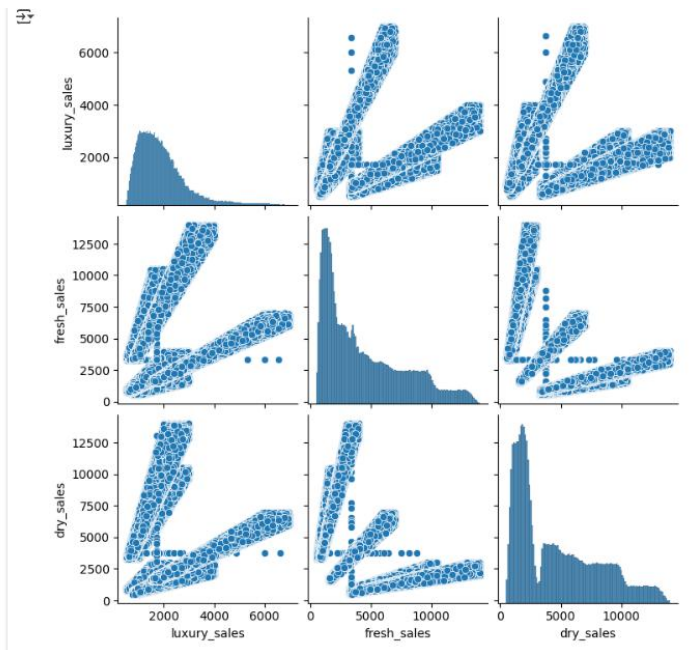
Correlation Heatmap



This correlation heatmap illustrates the relationship between luxury, fresh and dry sales categories. The correlation values range from -1 to 1 such that 1.00 (Red) is perfect positive correlation where both increase together, -1.00 (Blue) where one increases, the other decreases, and 0 with no correlation. This analysis shows a weak positive correlation (0.31) between luxury and fresh sales, indicating that customers who purchase luxury items may also buy fresh products, though the relationship is not strong. Similarly, a very weak correlation of

0.15 is there between luxury and dry sales, suggesting that luxury purchases do not significantly impact dry sales. However, a moderate negative correlation (-0.54) exists between fresh and dry sales, which means when fresh sales increase, dry sales tend to decrease. This shows that customers prefer either fresh or dry goods but do not usually purchase both in high quantities together. These insights can be really helpful in forming marketing strategies such as target marketing.

Pairplot



This pairplot shows the relationships between luxury sales, fresh sales and dry sales in different plots at once. In the diagonal (histograms) all three variables are right skewed. In the scatter plots there are positive correlations between all three types of sales which means customers who tend to spend more in one type of good tends to spend more in others. The scatter plots show distinct clusters, possibly indicating different customer segments within the sales types.

Customer segmentation Analysis and Labelling

To gain insights into customer purchasing behaviour in order to recognize the predefined customer segments, the dataset was grouped by cluster categories and the average sales values for luxury, fresh and dry sales were calculated.

```
# Group by 'cluster_category' and main sales columns
grouped_df = df.groupby('cluster_catgeory')[['luxury_sales', 'fresh_sales', 'dry_sales']].mean()
# Display the summary
print(grouped_df)
```

	luxury_sales	fresh_sales	dry_sales
cluster_catgeory			
1	1563.129381	2188.536814	8436.339507
2	1249.650229	6745.451160	1749.709650
3	4070.384286	4071.317814	4069.927554
4	1748.636760	1250.060977	6742.590765
5	3246.187056	3248.656666	3246.270241
6	2185.049856	8425.463958	1562.301899

This analysis provided a clear understanding of the spending patterns and trends across different customer groups. The results indicated distinct purchasing tendencies such as higher dry sales in cluster 1 and fresh sales in cluster 6, which helped in identifying customer preferences facilitating more targeted business strategies.

To enhance the interpretability, meaningful segment names were assigned to each cluster category based on the observed spending behaviours. A mapping dictionary was created to label clusters with suitable names such as “Bulk buyers”, “Luxury Seekers” and “Budget-Conscious Shoppers”. These labels were then intergrated into the dataset using the map() function creating a new column called “segment_name”. The updated dataset was saved for verification. This significantly improved the clarity of customer segmentation making it easier to develop personalized marketing strategies tailored to each group.

```
[ ] # Define segment names based on analysis
segment_mapping = {
    1: "Bulk Buyers",
    2: "Health-Conscious Buyers",
    3: "Luxury Seekers",
    4: "Budget-Conscious Shoppers",
    5: "Occasional Shoppers",
    6: "Everyday Shoppers"
}

# Create a new column with segment names
df["segment_name"] = df["cluster_catgeory"].map(segment_mapping)

# Save the updated dataset if needed
df.to_csv("segmented_customers.csv", index=False)
```

The first few rows of the dataset is displayed in order to confirm the updates. Each row now includes a segment name based on customer behaviour.

```
[ ] # Display the first few rows to verify
    print(df.head())
```

```
↵ Customer_ID outlet_city luxury_sales fresh_sales dry_sales \
0    10493832.0 Kelaniya      1209.60      756.00    5292.00
1    10178643.0 Moratuwa      1590.12     1060.08    6007.12
2    10513916.0 Wattala       2288.88     1481.04    9155.52
3    10334589.0 Wattala       2274.94     1739.66    9099.76
4    10458365.0 Kelaniya      2345.49     2069.55    9243.99

    cluster_catgeory      segment_name
0                4  Budget-Conscious Shoppers
1                1             Bulk Buyers
2                4  Budget-Conscious Shoppers
3                4  Budget-Conscious Shoppers
4                4  Budget-Conscious Shoppers
```

How ?

As mentioned above the buying patterns and trends were used to decide on the appropriate names for the customer segmentations.

Bulk buyers (cluster 1)

Highest dry sales (8,436) but moderate fresh & luxury sales indicate a preference for long-lasting and non-perishable goods. These customers likely buy in bulk, stocking up on essentials rather than making frequent smaller purchases.

Health-Conscious Buyers (Cluster2)

Highest fresh sales (6,745) with significantly lower dry & luxury sales. This suggests that these customers focus on fresh food items, perishable like vegetables and fruits, prioritizing healthy eating habits over bulk or luxury spending.

Luxury Seekers (Cluster 3)

High spending across all the categories approximately 4070 each suggests a balanced yet a luxury shopping behaviour, and in the EDA above it is shown that there are not many customers in cluster 3 which ensures these are luxury seeking customers. These customers are likely willing to spend more on the high-end products in all segments, making them ideal for luxury marketing.

Budget-Conscious Shoppers (Cluster 4)

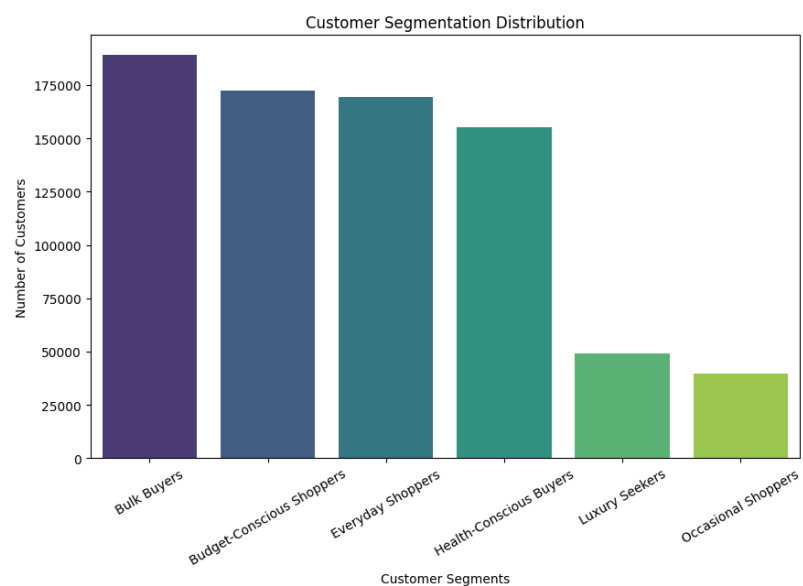
Relatively high dry sales (6,742) but low fresh (1,250) & luxury (1,748) sales indicate cost-saving behaviour. These customers prioritize non-perishable essentials over perishables.

Occasional Shoppers (Clusters 5)

Moderate spending with each category with an approximate of 3246 but with the smallest customer base. This suggests that they shop occasionally and maintain a balanced but lower frequency of purchases, unlike everyday or bulk buyers.

Everyday Shoppers (Cluster6)

Highest fresh sales (8,425) with moderate dry & luxury sales suggests frequent grocery shopping habits. These customers prioritize fresh food, indicating they shop regularly rather than in bulk



The above bar chart shows the number of customers in the the reamed predefined cluster categories.

Feature Engineering

Encoding Strategies

Data Preparation

```
[1] #Load Dataset using pandas library
import pandas as pd
df=pd.read_csv('/content/cleaned_train.csv')
df.head()
```

	Customer_ID	outlet_city	luxury_sales	fresh_sales	dry_sales	cluster_catgeory
0	10493832.0	Kelaniya	1209.60	756.00	5292.00	4
1	10178643.0	Moratuwa	1590.12	1060.08	6007.12	1
2	10513916.0	Wattala	2288.88	1481.04	9155.52	4
3	10334589.0	Wattala	2274.94	1739.66	9099.76	4
4	10458365.0	Kelaniya	2345.49	2069.55	9243.99	4

```
[ ] #Encoding categorical variables
from sklearn import preprocessing
label_encoder = preprocessing.LabelEncoder()
df['Encoded_outlet_city']= label_encoder.fit_transform(df['outlet_city'])
df.head()
```

	Customer_ID	outlet_city	luxury_sales	fresh_sales	dry_sales	cluster_catgeory	Encoded_outlet_city
0	10493832.0	Kelaniya	1209.60	756.00	5292.00	4	11
1	10178643.0	Moratuwa	1590.12	1060.08	6007.12	1	13
2	10513916.0	Wattala	2288.88	1481.04	9155.52	4	19
3	10334589.0	Wattala	2274.94	1739.66	9099.76	4	19
4	10458365.0	Kelaniya	2345.49	2069.55	9243.99	4	11

Label Encoding was applied to outlet_city to convert it into numerical values. Encoding was necessary to ensure compatability with machine learning models since ML models cannot directly intrepret text data. Models like Logistic Regression, KNN, XGBoost and Random Forest require numerical input. Encoding is required to transform these categories into numerical values. The technique used here is Label Encoding.

Standardization & Normalization

Features

First the features (x) and target (y) are defined. x (Features) contains independent variables that will be used to predict the target. y (Target) contains the dependant variables (cluster_category), which is what the model aims to predict.

```
[ ] #create a dataframe for Featuers (X) and Targets (y)
X = df[['Encoded_outlet_city', 'luxury_sales', 'fresh_sales', 'dry_sales']]
y = df['cluster_catgeory']
```

Feature Scaling

```
[ ] # Scaling

# Standardize input variables
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X1 = scaler.fit_transform(X)

# Normalize input variables
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
X2 = mms.fit_transform(X)
```

StandardScaler transforms the data into mean of 0 and a standard deviation of 1. It helps when features have different scales and ensures stable gradient descent in ML algorithms.

For Normalization MinMaxScaler scales values between 0 and 1. This is useful when dealing with data that should be bounded.

Splitting

```
from sklearn.model_selection import train_test_split

# Split X1 and y into training and testing sets
X1_train, X1_test, X2_train, X2_test, y_train, y_test = train_test_split(X1, X2, y, test_size=0.3, random_state=42, stratify=y)

print('Whole Data shape', df.shape)
print('X1_train shape', X1_train.shape)
print('X1_test shape', X1_test.shape)
print('X2_train shape', X2_train.shape)
print('X2_test shape', X2_test.shape)
```

Whole Data shape (774150, 7)
X1_train shape (541905, 4)
X1_test shape (232245, 4)
X2_train shape (541905, 4)
X2_test shape (232245, 4)

The dataset is splitted into Training & Testing such that 70% of the dataset is used to train the model while 30% is used to test the dataset. `random_state=42` ensures reproducibility. `stratify=y` is used to ensure that the class distribution `y` remains the same in both training and test sets.

The shape of the dataset is checked and it confirms that the dataset was split correctly. The dataset has 774150 samples and after the split 70% (541,905) goes to training and 30% (232,245) goes to testing. Splitting data helps in evaluating the model performance and prevents overfitting.

Model Development

Algorithms Considered

We experimented with a few models;

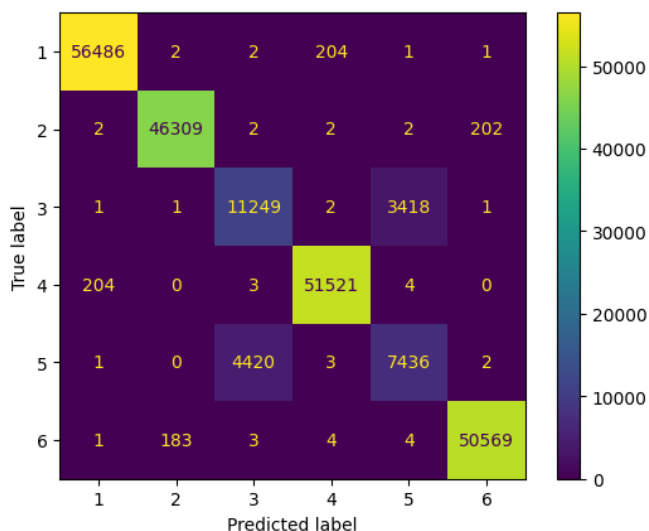
1. **Logistic Regression** - It was simple and interpretable. Works well if the relationship between features and target is linear
2. **K-Nearest Neighbours (KNN)** - Performed really well and could almost consider the best but it was slightly overfitting.
3. **Random Forest**- Provided high accuracy and important insights but is overfitting and slowed down the hyperparameter tuning. Hence decided to get rid of this model
4. **Naïve Bayes** – Both parameters underfitted in this model.
5. **XGBoost** – Best performing model due to its handling of complex data relationships but the model overfitted.

Model Selection & Performance

Best Model : Logistic Regression

The final model would depend on Performance Metrics, Computational Efficiency, Overfitting Handling and Interpretability.

The logistic regression model achieved an overall accuracy of around 96% , performing exceptionally well for most classes. The classification report and confusion matrix indicate that Classes 1,2,4 and 6 were have almost perfect classification with very few misclassifications, while category 3 and 5 showed lower performance. This suggests that these two classes have overlapping feature distributions, indicating the model struggles a little to differentiate these categories. However, this model neither overfit nor underfit.

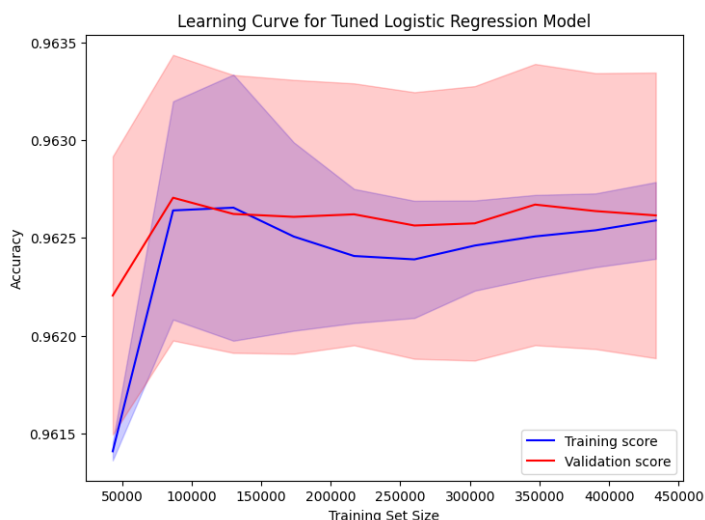


	Model	Precision	Recall	F1 Score	AUC Score \
0	Logistic Regression	0.962366	0.962475	0.962373	0.998314
1	KNN	0.999755	0.999755	0.999755	0.999864
2	Gaussian Naive Bayes	0.778820	0.770294	0.767695	0.969006
3	XGBoost	0.999759	0.999759	0.999759	0.999902
4	Tuned Logistic Regression	0.962517	0.962647	0.962466	0.998205
5	Tuned KNN	0.999755	0.999755	0.999755	0.999864
6	Tuned Naive Bayes	0.778820	0.770294	0.767695	0.538028
7	Tuned XGBoost	0.999759	0.999759	0.999759	0.999904

	Accuracy
0	0.962475
1	0.999755
2	0.770294
3	0.999759
4	0.962647
5	0.999755
6	0.770294
7	0.999759

The ROC curve analysis further supports this, with all classes achieving AUC scores close to 1.00 except class 5, which had a slightly lower AUC of 0.98. Additionally, the ROC and AUC curves scores demonstrate that the model has a strong ability to distinguish between classes. Hyperparameter tuning was used to optimize the model's performance.

Why choose logistic regression model to be the best model?



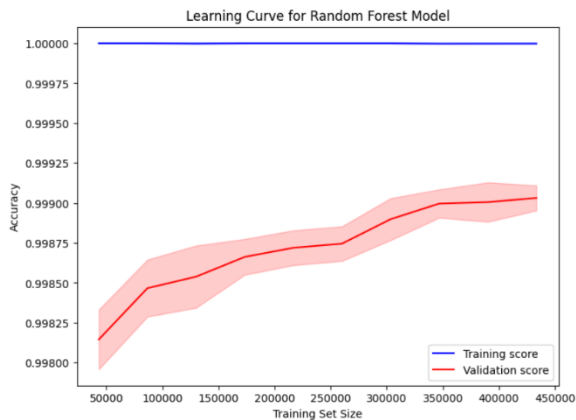
The logistic regression model has been determined to be the best choice based on its high accuracy (but not too high) stability, and generalization capability. The learning curve analysis confirms that this model is neither overfitting nor underfitting, as training and validation accuracy curves remain close throughout the learning process. Both curves stabilize around 96.25% indicating that the model is learning effectively without excessive variance or bias. The confusion

matrix and classification report further support this, showing excellent performance for most classes with only minor misclassifications in category 3 and 5. Given the observations, logistic regression is both computationally efficient and interpretable making it best-suited model for this problem.

Challenges & Solutions

Overfitting

There were indeed many challenges that we had to overcome as we developed the model. Even though experimented with many models they could not be used due to overfitting and underfitting.



This is one of the models that was overfitted and was not useful. Here the training accuracy remains at 100% and the gap between the training and validation accuracy indicates high variance. This model might be memorizing the training than generalizing.

Similarly, we had to face underfitting of models too. But we managed to create a more generalized model which was a Logistic regression model, both curves stabilizing around 96.25% accuracy.

Computational Constraints

During model training, most of the calculations were performed on the CPU. While logistic regression was the fastest to train, other models experienced slower execution due to limited computing power. To overcome this, we used Google Colab's GPU to train K-Nearest Neighbors and XGBoost models, which significantly improved processing speed. Additionally, when tuning hyperparameters, we used `n_jobs=-1` to speed up the process by using all available CPU cores.

Marketing Strategy Recommendations

The model we created is a sophisticated analytical model method to identify groups of similar customers in order to adopt a personalized marketing strategy tailored to individual customer preferences as an effort to enhance marketing strategies.

Marketing Strategy Recommendations based on customer segmentations

1. Bulk Buyers

- Providing wholesale deals and family packages
- These are the customers buying large quantities of goods specially dry items. They can possibly be small businesses, caterers or family households.
- They might value wholesale pricing and bulk discounts rather than individual promotions. A B2B model could also help retain them.
- For example;
 - Wholesale Membership
 - Family-sized packages of high consumption items

2. Health-Conscious Buyers

- Highlight organic and fresh food promotions.
- These are the customers who spend a lot on fresh vegetables and fruits, dairy and meat but not on dry or luxury items.
- These shoppers care about the quality and health benefits rather than price. Promoting organic, locally sourced and farm-fresh items will keep them loyal.
- For example;
 - “Farm Fresh Wednesdays” with discounts on farm products
 - Loyalty programs with exclusive discounts on organic and fresh foods

3. Luxury Seekers

- Target with exclusive offers, premium loyalty programs.
- Customers who spend heavily on luxury items and also spends on regular items.
- Since they prefer premium goods, standard discounts might not appeal to them. Instead, offering VIP memberships, exclusive previews of new luxury products and premium services like free delivery of high end items would keep them engaged.
- For example;
 - Early access to imported wines, chocolates or luxury skincare
 - A loyalty program where higher spendings unlocks personalized assistance services

4. Budget-Conscious

- Offer-bulk buying incentives and discounts on dry goods.
- These are the customers who mainly buy dry goods which are long-shelf items like rice, canned food, and pasta and spend less on luxury and fresh items.
- They prefer cost-effective options and might be price-sensitive. Encouraging bulk purchases will drive more revenue while helping them save.
- For example :
 - “Buy more, Save more” offers
 - Subscriptions-based grocery plans

5. Occasional Shoppers

- Send re-engagement offers and reminders.
- These customers shop frequently or have switched to competitors.
- They need reminders, incentives or better deals to bring them back.
- Sending targeted discounts or personalized messages could help them retain.
- For example;
 - Re-engagement emails/texts like, “We haven’t seen you in a while! Here’s a 10% discount on your next purchase”.
 - Personalized offers based on past purchases

6. Everyday shoppers

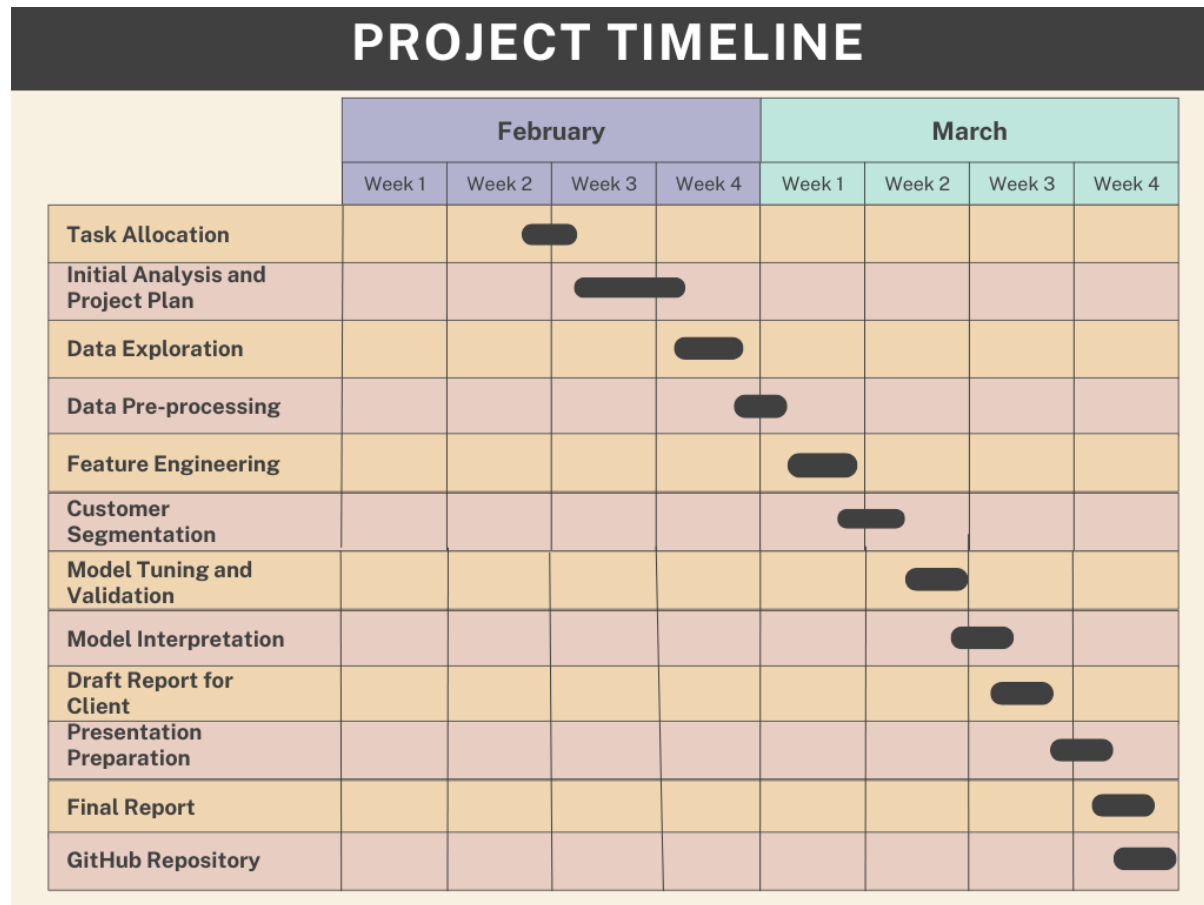
- Promote discounts on household essentials.
- These customers are not heavily brand loyal but are likely to return for convenience and affordability.
- Offering targeted discounts on their regular grocery list ensures they chose this supermarket over the competitors.
- For example;
 - Weekly Essential Saver discounts on frequently bought items like rice, bread, dairy and snacks.
 - Bundle deals.

Project Plan

Work Breakdown Structure

Task	Description	Timeline	Member
Task Allocation	Assigning tasks to team members	15 Feb – 17 Feb	Project Manager
Initial Analysis and Project Plan	Creating project plan and scope	18 Feb – 24 Feb	Project Manager
Data Exploration	Exploring dataset for insights	25 Feb – 27 Feb	Data Analyst
Data Pre-processing	Cleaning and preparing data	28 Feb – 02 Mar	Data Engineer
Feature Engineering	Creating new features from data	03 Mar – 07 Mar	Data Scientist
Customer Segmentation	Clustering customers based on data	08 Mar – 11 Mar	ML Engineer
Model Tuning and Validation	Improving model performance	11 Mar – 14 Mar	ML Engineer
Model Interpretation	Understanding model predictions	15 Mar – 17 Mar	Business Analyst
Draft Report for Client	Preparing first draft of report	18 Mar – 20 Mar	Report Writer
Presentation Preparation	Creating presentation slides	21 Mar – 24 Mar	Presentation Lead
Final Report	Finalizing project report	25Mar – 28 Mar	Report Writer
GitHub Repository	Organizing code repository	29 Mar – 31 Mar	GitHub Manager
Submit Final Deliverables	Submitting all project materials	3 April	Project Manager

Gantt chart



Project team

Nethmi Maleesha – Team leader/ Project Manager / Data Scientist

Sithuli Kothalawala – Data Engineer / Report Writer

Shahma Silmy – Data Analyst / Business Analyst / Presentation Lead

Ruchintha Dias – ML Engineer / GitHub Manager

GitHub Link

https://github.com/InsightMart/DSPL_project.git

Canva Link

https://www.canva.com/design/DAGibHqFpel/dSf70Kn0rvHtpPHQKVfslw/view?utm_content=DAGibHqFpel&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utm_id=hf718cad6c8

TEAM MEMBERS

IIT ID	UOW ID	UOW Username	Blackboard Name
20230543	20521336	W2052133	Mallawa Maleesha
20230336	20521288	W2052128	Sithuli Kothalawala
20221560	20521659	W2052165	Ahamed Shahma
20232724	20844558	W2084455	Ruchintha Dias